

Functions and Reusable Calculations

Engineering Computation with Python — Chapter 5

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

1 What Is a Function?

2 Pre-Defined Functions

3 Writing Your Own Functions

4 Variable Scope

5 Testing Functions

6 Summary

- 1** What Is a Function?
- 2** Pre-Defined Functions
- 3** Writing Your Own Functions
- 4** Variable Scope
- 5** Testing Functions
- 6** Summary

So far every program has been written as one continuous script. That works while a calculation is short, but engineering calculations are **repeated** — the same conversion applied to twenty readings, the same formula checked at a dozen operating points.

A function

Packages a calculation once, gives it a name, and lets the rest of the program call it as often as needed.

- Makes code cleaner and easier to read
- Improves reusability and maintainability
- Hides complex implementation details

Aspect	Script	Function
Usage	Executes commands directly	Defines a reusable task
Input/output	Global variables, user input	Arguments, return values
Scope	Uses global variables	Uses local variables

The key shift

A script runs top to bottom, once. A function is written once and **called** as many times as needed, with different inputs each time.

Type	Description	Example
Built-in	Predefined in Python	<code>print(), len()</code>
Library	From external modules	<code>math.sqrt()</code>
User-defined	Created by the programmer	<code>def greet():</code>
Nested (local)	Defined inside another function	see below

This chapter covers all four, with the focus on **writing your own**.

1	<u>What Is a Function?</u>
2	<u>Pre-Defined Functions</u>
3	<u>Writing Your Own Functions</u>
4	<u>Variable Scope</u>
5	<u>Testing Functions</u>
6	<u>Summary</u>

Built-in Functions: Always Available

Built-in functions need **no import** at all:

```
readings = [12.0, 11.5, 13.2, 10.8]    # volts

print(len(readings))                  # 4
print(max(readings))                  # 13.2
print(min(readings))                  # 10.8
print(sum(readings) / len(readings))  # 11.875 -- the mean
```

`sum(x)/len(x)` is worth remembering: an average is nothing more than a total divided by a count. Chapter 14 replaces this with `np.mean()`.

Python rounds a value ending in exactly .5 to the nearest **even** number, not always upward:

```
print(round(2.5))    # 2, not 3  
print(round(3.5))    # 4
```

Banker's rounding

This exists so that rounding a large set of measurements does not accumulate a systematic upward bias. Correct behaviour, not a defect — but it surprises almost everyone the first time.

Library functions become available through **importing**:

```
import math
print(math.sqrt(16))           # 4.0

import numpy as np
array = np.array([1, 2, 3])

import random
choice = random.choice(['apple', 'banana', 'cherry'])
```

Every library used in this course — math, numpy, pandas, random — follows this same import pattern.

1 What Is a Function?

2 Pre-Defined Functions

3 **Writing Your Own Functions**

4 Variable Scope

5 Testing Functions

6 Summary

A user-defined function uses `def`, a name, parameters, a colon, and an indented body. `return` sends a result back.

```
def function_name(parameter1, parameter2):  
    """Description of what the function does"""  
    # Code block  
    return result
```

```
def add(a, b):  
    return a + b  
  
print(add(3, 7)) # 10
```

Five Steps to Define a Function

1. **Define:** def keyword, name, parentheses
2. **Parameters** (optional): inputs inside the parentheses
3. **Colon ::** starts the function body
4. **Body:** indented code
5. **Return** (optional): sends back a result

Remember

A function that never uses return still runs fine — it simply returns None to the caller.

The simplest shape: one value in, one value out.

```
def square(number):  
    return number ** 2  
  
print(square(4))  # 16
```

The four input/output shapes — single/single, single/multiple, multiple/single, multiple/multiple — are all equally valid in Python. The next few slides show each.

Multiple return values are packed together as a **tuple**:

```
def analyze_number(num):  
    is_even = num % 2 == 0  
    square = num ** 2  
    return is_even, square
```

```
even, sq = analyze_number(5)  
print(even, sq)  
# False 25
```

Returning several values at once is idiomatic in Python — and convenient for engineering results.

```
def calculate(a, b):  
    addition = a + b  
    subtraction = a - b  
    multiplication = a * b  
    division = a / b if b != 0 else None  
    return addition, subtraction, multiplication, division  
  
s, d, p, q = calculate(8, 2)  
print(s, d, p, q)  
# 10 6 16 4.0
```

Any combination of inputs and outputs is possible — choose the shape that matches the calculation.

The caller can choose to use only the outputs it needs:

```
def motion(t):  
    distance = [0.0001 * i**2 for i in t]  
    acceleration = [0.0025 * i**2 for i in t]  
    return distance, acceleration  
  
distance, acceleration = motion(t)    # both  
distance = motion(t)[0]              # index into the tuple  
distance, _ = motion(t)              # discard the 2nd value  
_, acceleration = motion(t)          # discard the 1st value
```

The underscore `_` is a common convention for “I don’t need this value.”

A function defined **inside** another function is a local function — accessible only from within the enclosing function.

```
def process_order(quantity, price_per_item):  
    def calculate_total(qty, price):          # local function  
        total = qty * price  
        if total > 100:  
            return total * 0.9 # 10% discount  
        return total  
  
    total_price = calculate_total(quantity, price_per_item)  
    return f"Total Price: ${total_price:.2f}"  
  
print(process_order(10, 15))  
# Total Price: $135.00
```

Multiple Local Functions and Default Arguments

```
def process_order(quantity, price_per_item):  
    def calculate_total(qty, price):  
        total = qty * price  
        return total * 0.9 if total > 100 else total  
  
    def apply_tax(amount, tax_rate=0.05): # default argument  
        return amount * (1 + tax_rate)  
  
    total_price = apply_tax(calculate_total(quantity, price_per_item))  
    return f"Total after tax: {total_price}"
```

apply_tax has a **default argument**: if tax_rate is not supplied, it defaults to 0.05.

A **docstring** documents what a function does, what it expects, and what it returns.

```
def calculate_circle_area(radius):  
    """  
    Calculate the area of a circle given its radius.  
  
    Parameters:  
        radius (float): The radius of the circle.  
    Returns:  
        float: The area of the circle.  
    """  
    return math.pi * radius ** 2  
  
help(calculate_circle_area)
```

The difference between a function a colleague can use, and one they must reverse-engineer.

1	<u>What Is a Function?</u>
2	<u>Pre-Defined Functions</u>
3	<u>Writing Your Own Functions</u>
4	<u>Variable Scope</u>
5	<u>Testing Functions</u>
6	<u>Summary</u>

Every variable lives within a certain **scope** — the region of code where it can be seen or accessed.

Local

Defined inside a function. Only works there. Deleted when the function finishes.

Global

Defined outside all functions. Accessible throughout the program, including inside functions.

This distinction causes some of the most confusing bugs beginners encounter — worth understanding carefully.

```
def greet():  
    name = "Alice" # local variable  
    print(name)  
  
greet()  
print(name) # NameError: name is not defined
```

What happened

name only exists while greet () is running. Once the function finishes, it is **deleted from memory** – the outer print(name) has nothing to find.

```
name = "Bob"  # global variable

def greet():
    print(name)  # Python finds it in the global scope

greet()        # Bob
print(name)    # Bob
```

What happened

There is no local name inside `greet()`, so Python automatically looks outward and finds the global one.

When a function **assigns** to a name, Python treats that name as local for the *entire* function body:

```
count = 0 # global

def increase():
    count = 5 # creates a NEW local variable
    print("Inside:", count)

increase()
print("Outside:", count)
```

```
Inside: 5
Outside: 0
```

The local count **shadows** the global one – but only inside the function.

Modifying a Global: The Wrong Way

```
count = 0 # global variable

def increase():
    count = count + 1 # tries to change count
    print(count)

increase()
```

```
UnboundLocalError: local variable 'count'
referenced before assignment
```

Why

Python sees `count = ...` and assumes `count` is local for the **whole** function – then tries to read it before it has been created.

Modifying a Global: The Right Way

Use the `global` keyword to tell Python which count you mean:

```
count = 0 # global variable

def increase():
    global count # use the global one, not a new local
    count = count + 1
    print(count)

increase()
print(count)
```

```
1
1
```

Rule of thumb

Declare `global` where you intend to **change** the value, not merely read it. Prefer parameters and return values where you can.

1 What Is a Function?

2 Pre-Defined Functions

3 Writing Your Own Functions

4 Variable Scope

5 **Testing Functions**

6 Summary

A test is simply a call whose result you already know, checked with `assert`:

```
def add(a, b):  
    return a + b  
  
assert add(2, 3) == 5  
assert add(-1, 1) == 0  
assert add(0, 0) == 0
```

Pass vs. fail

If all assertions hold, nothing is printed. If one fails, Python stops and tells you **exactly which case** broke.

Type	Example	Why important
Typical case	<code>add(2, 3)</code>	Normal usage
Edge case	<code>add(0, 0)</code>	Limits or boundaries
Negative case	<code>add(-5, 10)</code>	Behaviour with negatives
Unusual input	<code>add(999999, 1)</code>	Large values

Good test vectors cover a **range** of situations, not just the case you expect to work.

Worked Example: An Off-by-One Bug

Children aged 12 and under get a discount. This implementation has a subtle bug:

```
def legoland_price(age, base_price=50.0):  
    if age < 12:  # BUG: 12-year-olds should ALSO get the discount  
        return base_price * 0.5  
    return base_price  
  
assert legoland_price(11) == 25.0      # passes  
assert legoland_price(12) == 25.0     # FAILS!  
assert legoland_price(13) == 50.0     # passes
```

The fix

Change `age < 12` to `age <= 12`. The failing assertion at the **boundary** (age 12) is exactly what revealed the bug.

1	<u>What Is a Function?</u>
2	<u>Pre-Defined Functions</u>
3	<u>Writing Your Own Functions</u>
4	<u>Variable Scope</u>
5	<u>Testing Functions</u>
6	<u>Summary</u>

- A **function** packages a calculation under a name so it can be called repeatedly; a **script** executes once.
- Python has **built-in** functions (`print()`, `len()`), **library** functions (`math.sqrt()`), and **user-defined** functions you write with `def`.
- A function can take any number of inputs and return any number of outputs — returning several values as a tuple is idiomatic Python.
- Variables inside a function are **local** and vanish when it returns; this isolation prevents a function from silently corrupting the rest of the program.

- **Global** variables are visible everywhere, but assigning to a name inside a function creates a *new local* variable unless you declare `global` first.
- A **docstring** documents a function for the next person (often yourself) who has to use it.
- Test with `assert` and good **test vectors** — typical, edge, negative, and unusual cases, especially at boundaries.

Looking ahead

Every function here took one value in and returned one value out. A real instrument logs hundreds of readings — Chapter 6 introduces the **array**, which computes on many values at once.

Next: Arrays and Vectorized Computation